

Assignment-II

Pratik Beniwal

23-06-2026

Part I: Short Answer Questions

Q1: The query returns 0 because `NULL` represents an unknown value, not "zero" or "empty". writing `phone_number = NULL` asks if an unknown equals another unknown, which evaluates to `UNKNOWN`. The `WHERE` clause only keeps `TRUE` results, so no rows are selected.

Correct version:

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

generally, `NULL` is excluded from aggregate functions (like `SUM` and `AVG`) and comparisons because we cannot perform math or logical checks on unknown values. comparisons with `NULL` always return `UNKNOWN` rather than `TRUE` or `FALSE`.

Q2: yes, it is redundant. the `GROUP BY department` clause already ensures each department appears only once. since `DISTINCT` only removes duplicate rows, and duplicates are already gone, it does nothing useful here.

Q3: If a `LEFT JOIN` yields more rows than an `INNER JOIN`, it means some customers have never placed orders.

- **INNER JOIN:** only includes customers with at least one matching order.
- **LEFT JOIN:** includes all customers, filling order details with `NULLs` for those with zero orders.

Q4: using `GROUP BY department` groups all employees in a department into one row. `AVG(salary)` works by condensing salaries into a single average. however, `employee_name` is neither aggregated nor grouped. since SQL has no rule to pick one name from many, it rejects the query to avoid returning an arbitrary value.

Correct version:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

Q5: we get an error because aggregate functions like `AVG` cannot be used inside a `WHERE` clause. `WHERE` filters rows **before** grouping occurs. to filter based on an aggregate, we must use `HAVING`, which evaluates **after** groups are formed.

Correct version:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6:

- **Non-correlated:** products priced above the overall average price. the overall average is a single, fixed number calculated from the whole table. the subquery only needs to run once.
- **Correlated:** employees earning more than their own department's average. Here, "the average" changes depending on the specific employee we are evaluating. The subquery must run repeatedly for every employee, potentially slowing down large databases.

Part II: Long Answer Question

Part A - Customers who have never placed an order

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN "Order" o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Explanation: A standard INNER JOIN only shows customers who have placed orders. A LEFT JOIN includes every customer. by adding WHERE o.order_id IS NULL, we filter the list to show exactly the people who have blank order records.

Part B - Average order value per restaurant

```
SELECT ot.restaurant_id, AVG(ot.order_value) AS avg_order_value
FROM (
    SELECT o.order_id, o.restaurant_id, SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi ON o.order_id = oi.order_id
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY o.order_id, o.restaurant_id
) AS ot
GROUP BY ot.restaurant_id
HAVING COUNT(*) > 5;
```

Explanation: first, we calculate the total cost of each individual order in the inner query. then, the outer query averages those totals for each restaurant. this two-step process prevents large orders with many cheap items from skewing the math.

Part C - Restaurants beating the platform average

```
SELECT b.restaurant_id, b.avg_order_value
FROM (
    SELECT ot.restaurant_id, AVG(ot.order_value) AS avg_order_value
    FROM (
        SELECT o.order_id, o.restaurant_id, SUM(mi.price * oi.quantity) AS order_value
        FROM "Order" o
        JOIN OrderItem oi ON o.order_id = oi.order_id
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY o.order_id, o.restaurant_id
    ) AS ot
    GROUP BY ot.restaurant_id
    HAVING COUNT(*) > 5
) AS b
```

```

WHERE b.avg_order_value > (
    SELECT AVG(all_orders.order_value)
    FROM (
        SELECT SUM(mi.price * oi.quantity) AS order_value
        FROM "Order" o
        JOIN OrderItem oi ON o.order_id = oi.order_id
        JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY o.order_id
    ) AS all_orders
);

```

Explanation: the overall platform average is just one fixed number that applies to everyone. since it doesn't change from restaurant to restaurant, the subquery is non-correlated and only needs to be calculated once.

Part D - Most ordered menu item per restaurant

```

SELECT restaurant_id, item_id, item_name, total_qty
FROM (
    SELECT mi.restaurant_id, mi.item_id, mi.item_name, SUM(oi.quantity) AS total_qty,
           RANK() OVER (PARTITION BY mi.restaurant_id
                        ORDER BY SUM(oi.quantity) DESC) AS rnk
    FROM OrderItem oi
    JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
) AS ranked
WHERE rnk = 1;

```

Explanation: first, we sum the total quantity of each item sold per restaurant. Then, we use a window function (RANK) to rank items inside each restaurant. This is superior to using MAX() because it handles ties cleanly and retains the item's name in the output.

Part III: Normalization

Functional Dependencies:

- student_id -> student_name
- course_id -> course_name, instructor_id
- instructor_id -> instructor_name, instructor_office
- (student_id, course_id) -> grade, enrollment_date

Is it in 1NF? yes. A table is in First Normal Form when every cell holds just one atomic piece of data. Every column here holds a single value per row, satisfying 1NF.

2NF Violation: our primary key is (student_id, course_id). however, several columns depend on only one half of that key (e.g., student_name depends only on student_id). this creates a partial dependency. if the only student in a course drops out and we delete their row, we lose all information about the course itself.

3NF Violation: we have a transitive dependency: course_id -> instructor_id -> instructor_name. The instructor's name depends on instructor_id, not the primary key. If an instructor changes offices, we must update every enrollment row tied to them, risking update anomalies.

3NF Decomposition:

- **Student**(student_id, student_name) — split out because student_id -> student_name.
- **Course**(course_id, course_name, instructor_id) — split out because course_id -> course_name, instructor_id.
- **Instructor**(instructor_id, instructor_name, instructor_office) — split out because instructor_id -> instructor_name, instructor_office, removing the transitive dependency.
- **Enrollment**(student_id, course_id, grade, enrollment_date) — keeps only what depends on the full key.

Part IV: Design Justification

LEFT JOIN vs NOT EXISTS: both methods successfully find customers who never ordered. We selected the LEFT JOIN because:

- the IS NULL logic makes the "no match" condition immediately obvious.
- we avoided NOT IN because a single NULL value in the order table can break the query and return zero results.

Normalization Trade-offs: normalizing the Enrollment table eliminates data anomalies but increases query complexity.

- **Pros:** prevents data duplication and update errors.
- **Cons:** requires joining multiple tables to view complete records (e.g., joining four tables just to see a student's name, course, grade, and instructor's office), which increases database workload.
- **Real-world application:** systems often intentionally retain some denormalization (like in reporting/analytics tables) to keep read operations simple and fast at the cost of minor duplication.